Lecture Notes for

Neural Networks I

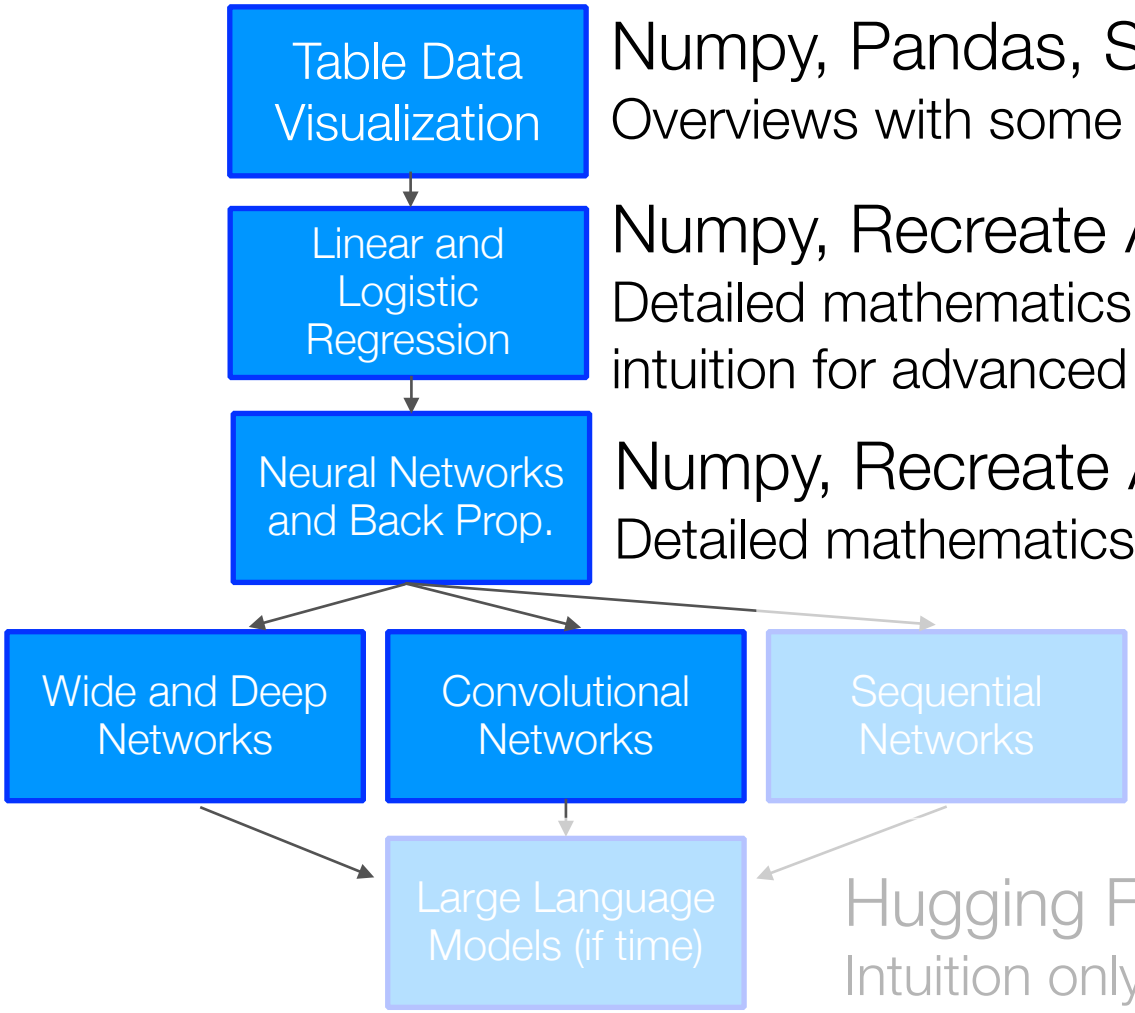


More CNN Architectures

Class logistics and Agenda

- Logistics:
  - Next Week
  - Grading Update
- Agenda:
  - More Advanced CNNs

Class Overview, by topic



Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

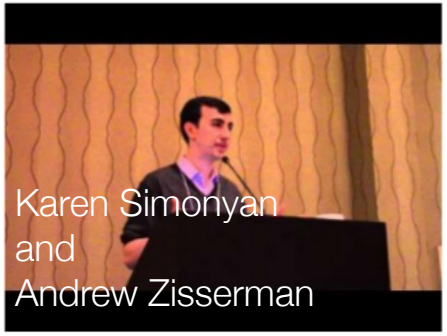
Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Hugging Face
Intuition only

Review: Transfer Learning Only



Types of CNN, 2013



UNIVERSITY OF OXFORD

Google DeepMind

- Oxford **VGG Net** (Visual Geometry Group)
- Major contributions:
  - small cascaded kernels
  - way more layers (19 versus ~7)
  - *"emulates" biology "better"* 😒
  - trained on NVIDIA GPUs for 2-3 weeks

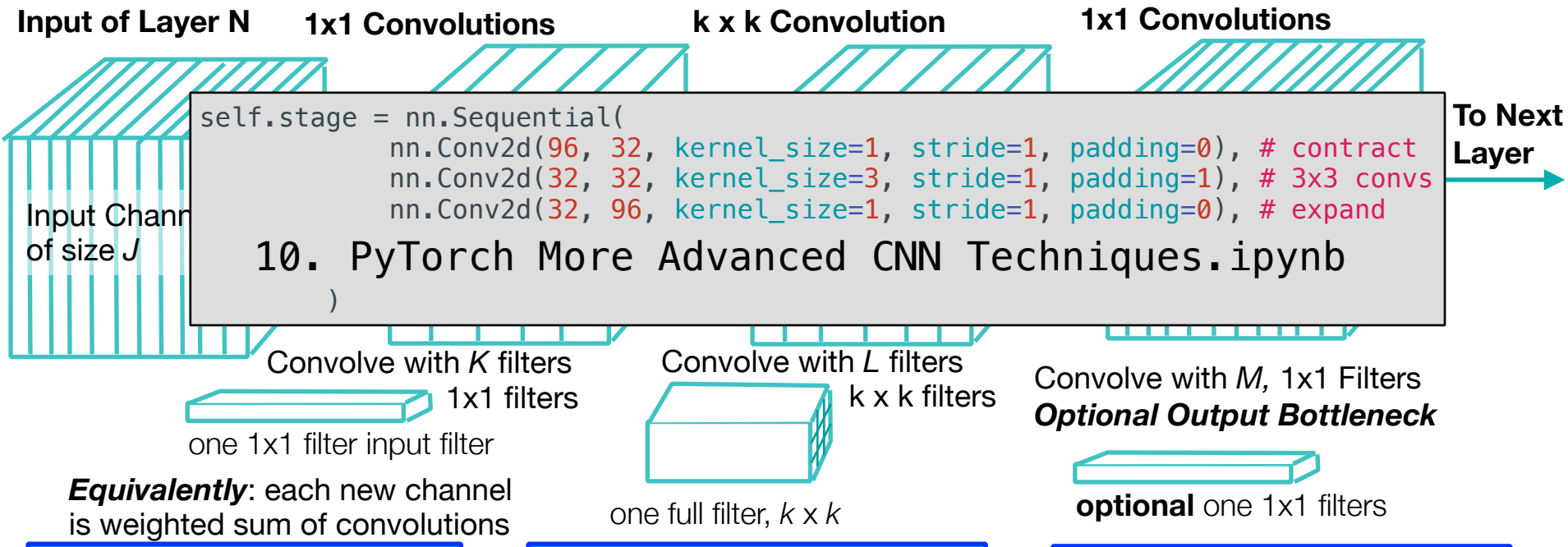
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

http://www.robots.ox.ac.uk/~vgg/practicals/cnn/

2014 Network in Network, expanded view



Equivalently: each new channel is weighted sum of convolutions

Reduce Channels using Lightweight 1x1 Filters
Wide ➡ Narrow

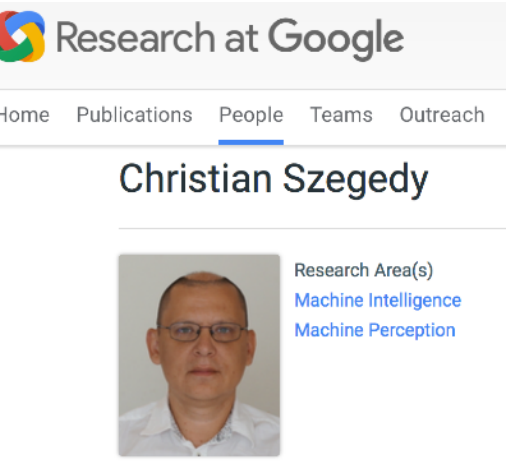
Larger k x k Filters on Reduced Channels
Narrow ➡ Narrow, fewer params

Expand with lightweight Filters, *(if desired)*
Narrow ➡ Wide

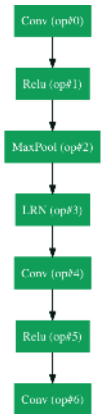
Example: Train 32 filters with kernel size of 3x3 (x256 input channels)(outputs size of 32)

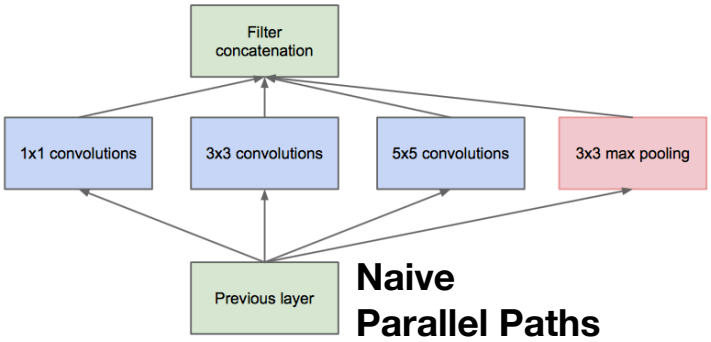
- No reduction: (32 x 3 x 3 x 256) = **73,728** (no bottleneck)
- Reduction to 128: (128 x 1 x 1 x 256) + (32 x 3 x 3 x 128) = **69,632** (reduce 128)
- Reduction to 64: (64 x 1 x 1 x 256) + (32 x 3 x 3 x 64) = **34,816** (reduce 64)

Types of CNN, 2014 (parallel pathways)

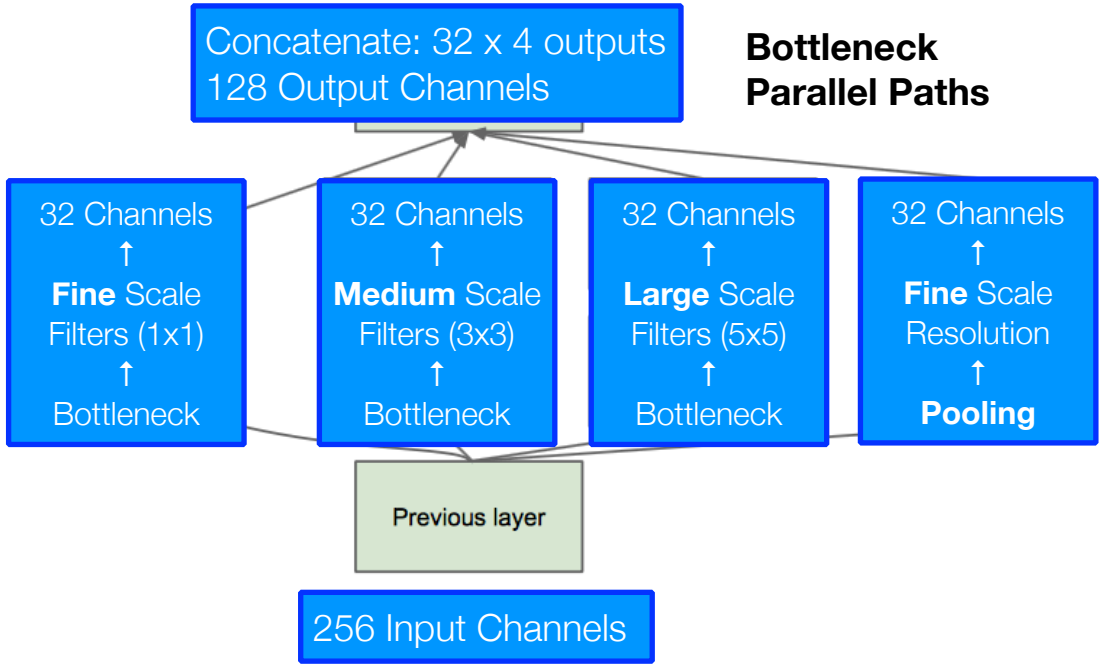


- **GoogLeNet**
  - or **Inception V1**
- Major contribution:
  - multi-scale filtering
  - parallel NiN





Naive Parallel Paths



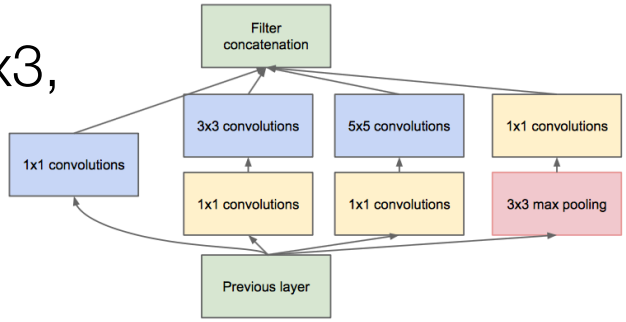
Bottleneck Parallel Paths

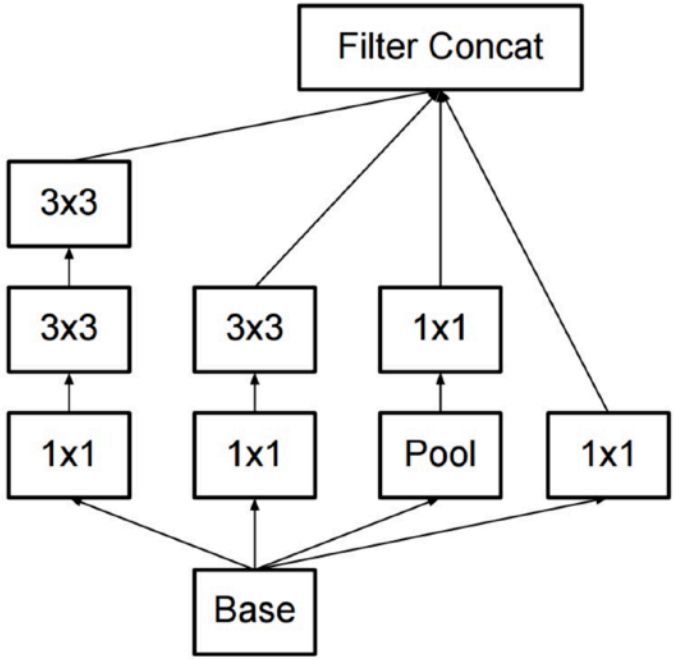
https://arxiv.org/pdf/1409.4842.pdf

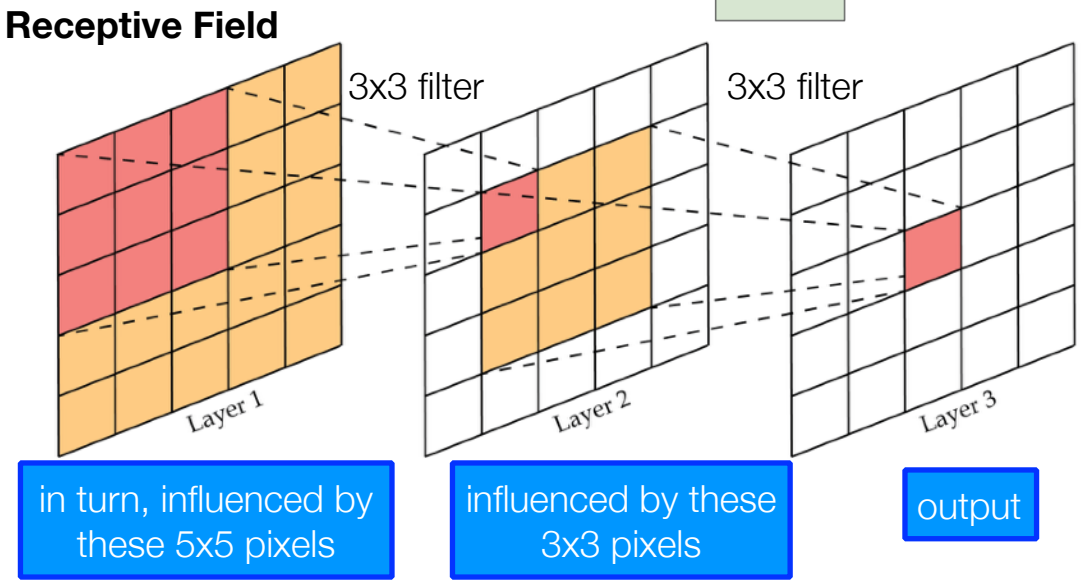
Types of CNN, 2015 February and December



- **Inception V2**,
  - Inception V1 with different normalization
- **Inception V3**:
  - replace 5x5 with multiple 3x3, maintains receptive field







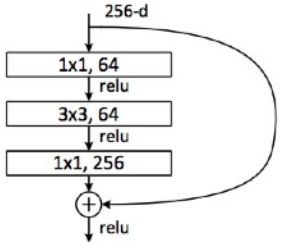
https://arxiv.org/pdf/1512.00567.pdf

https://medium.com/@rekalantar/receptive-fields-in-deep-convolutional-networks-43871d2ef2e9

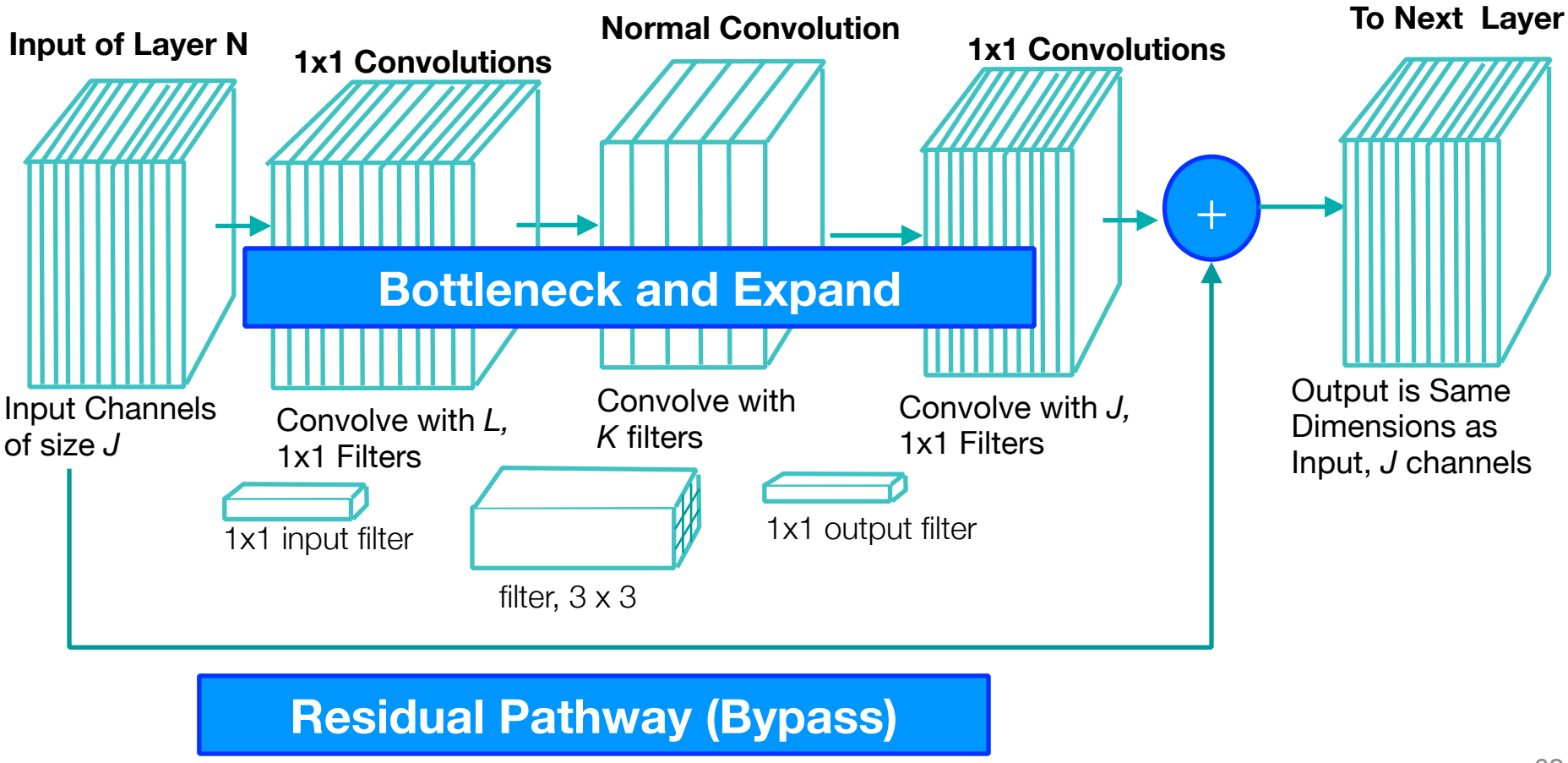
Types of CNN, 2015 December

Microsoft Research

- **ResNet,** Major Contributions:
  - bypass pathways
  - "bio-plausible" with feedback 😒



Back Propagation: Two paths, including one without ANY operations that cause the gradient to vanish…

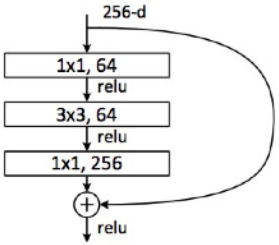


Residual Paths, in Code

In Model Class:

```
self.resblock1 = nn.Sequential(
            nn.Conv2d(256, 64, kernel_size=1, stride=1, padding=0),
            nn.Conv2d(64,  64, kernel_size=3, stride=1, padding=1),
            nn.Conv2d(64, 256, kernel_size=1, stride=1, padding=0),
        )

self.resblock2 = nn.Sequential(
            nn.Conv2d(256, 64, kernel_size=1, stride=1, padding=0),
            nn.Conv2d(64,  64, kernel_size=3, stride=1, padding=1),
            nn.Conv2d(64, 256, kernel_size=1, stride=1, padding=0),
        )
```



In Forward Method:

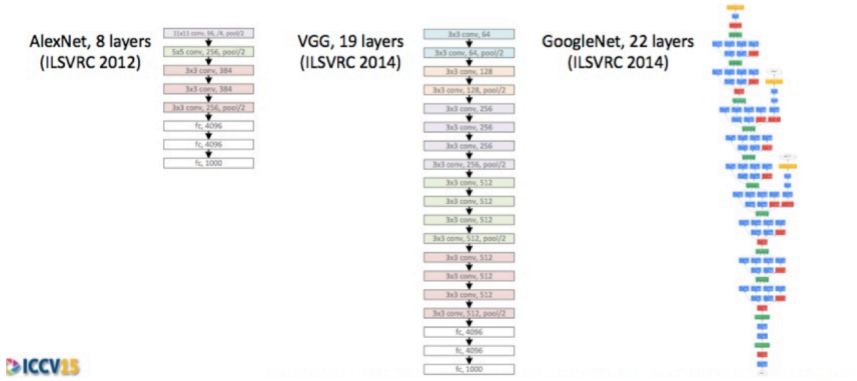
```
residual = x
x = self.resblock1(residual)
x = self.RELU( x + residual )

residual = x
x = self.resblock2(residual)
x = x + residual
```

10. PyTorch More Advanced CNN Techniques.ipynb

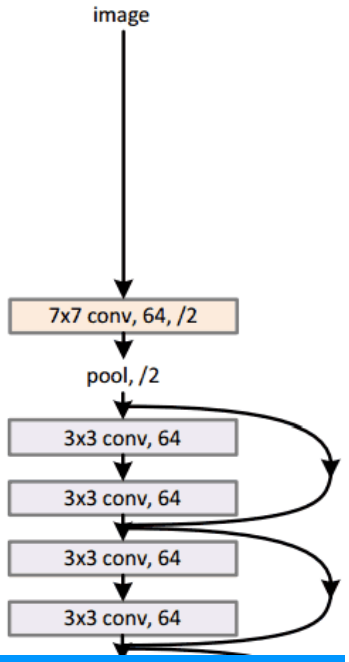
How big are these networks?

How big are these networks?



ResNet, 152 layers
(ILSVRC 2015)

34-layer residual



Transition Period in Convolutional Networks

- 2012 - 2017:
  - Add more layers! 🤪
  - How can we train it even deeper? 🤓
  - Can we run out of memory? Let's try! 🙈
- 2017-2021:
  - How can we get similar performance with reduced parameters? 🧐
  - How should the number of parameters scale for competing resource? Is there an optimum scaling for a given set of resources? 📉
- 2021 - present: Vision Transformers and Conv

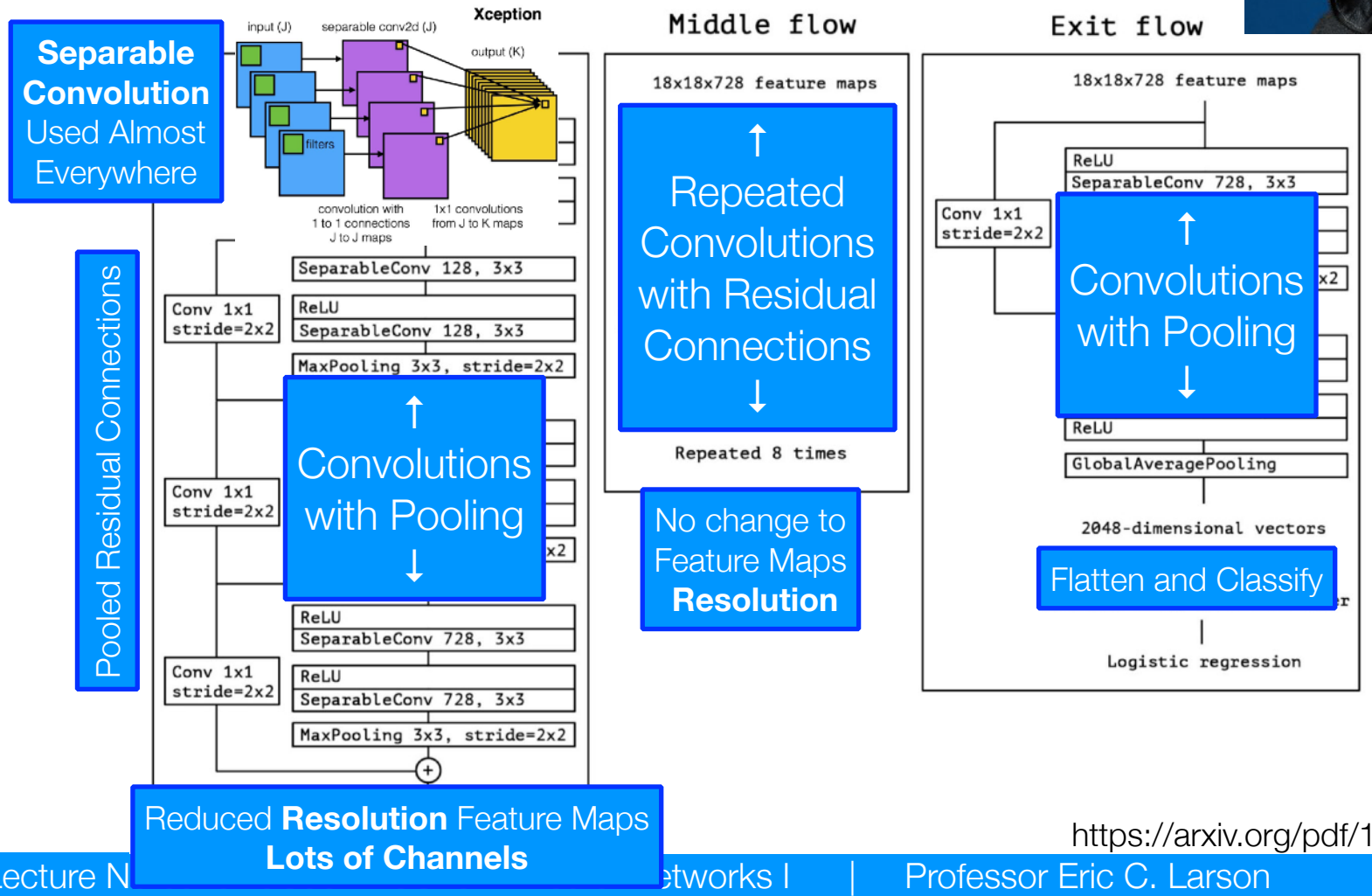
Types of CNN, 2017



Francois Chollet
Google

Xception · Major Contributions:

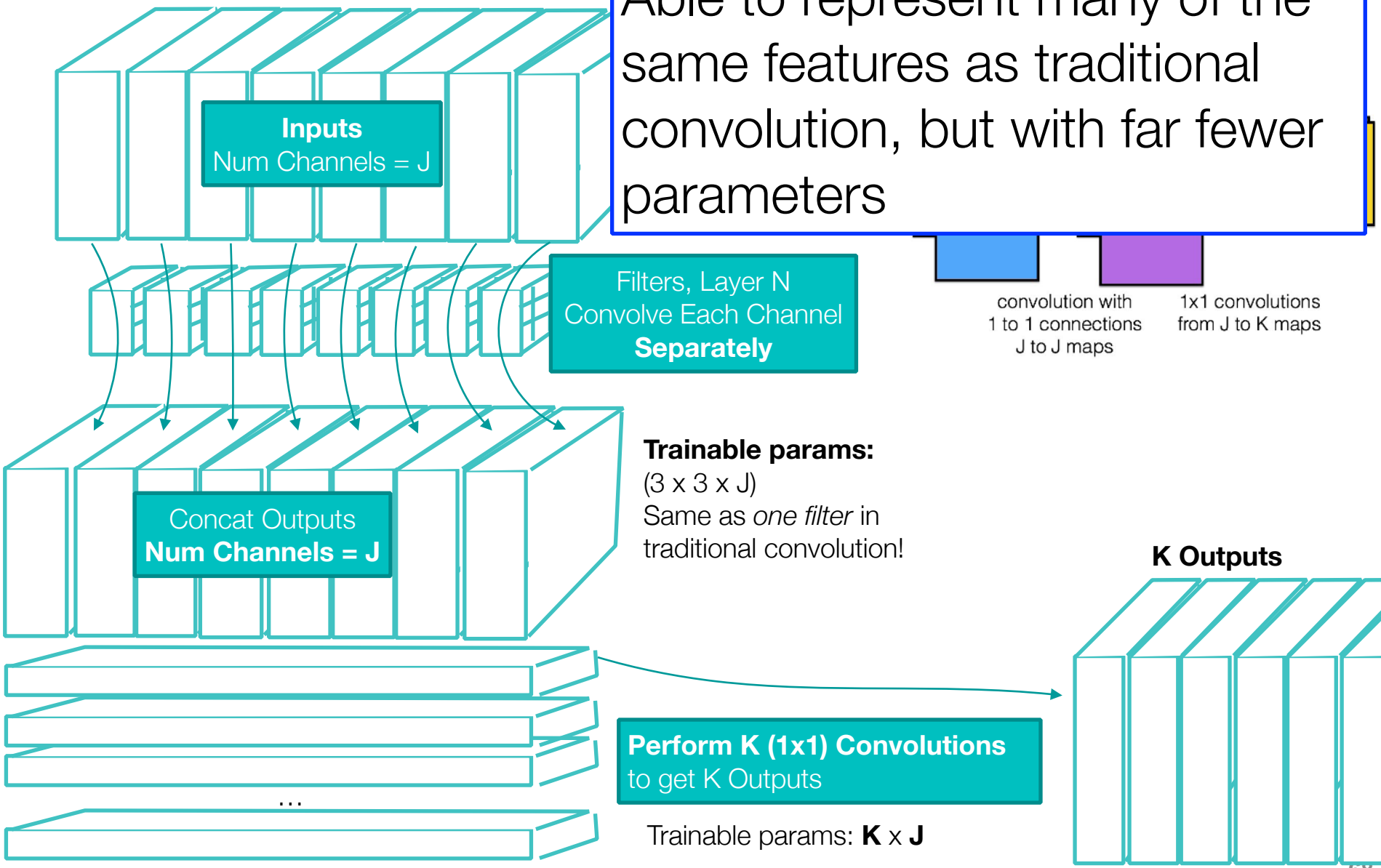
- combining branching / residual blocks
- separable convolutions (fewer trainable params)



https://arxiv.org/pdf/1610.02357.pdf

Separable Convolut

Separable Convolution:
Able to represent many of the same features as traditional convolution, but with far fewer parameters

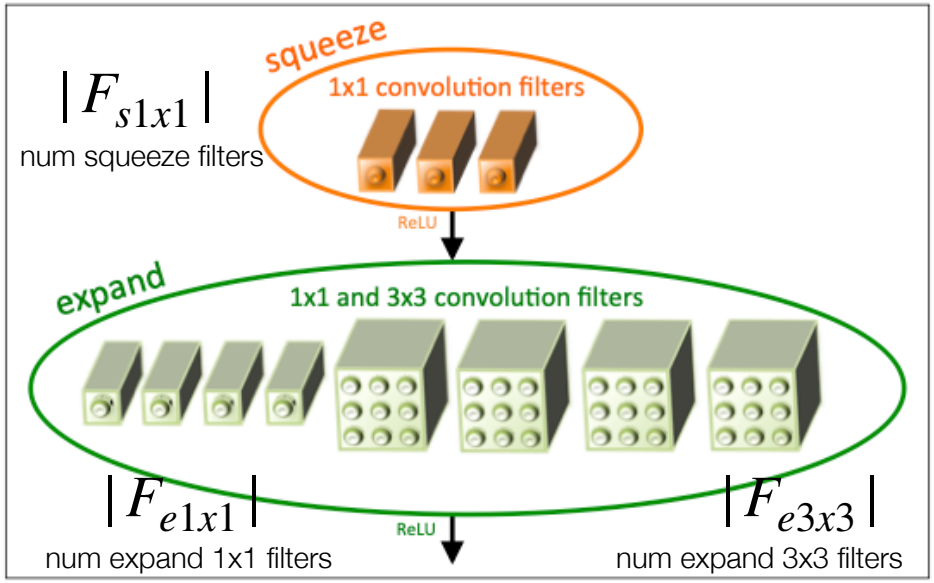


Separable Convolution

- Which is true regarding Separable Convolution:
- A. Each output convolution is a weighted sum of a channels from a “per-channel” convolution
- B. The effective number of trainable parameters is significantly reduced
- C. Each convolution is a $k \times k \times 1$ convolution with input that is a single channel from the input activations
- D. All of the above

SqueezeNet (2018)

- How to squeeze and expand in each layer, with residual pathways?
  - Decide bottleneck size from a ratio, *what ratio is best?*
  - Use mostly 1x1 filters, *how many versus 3x3?*

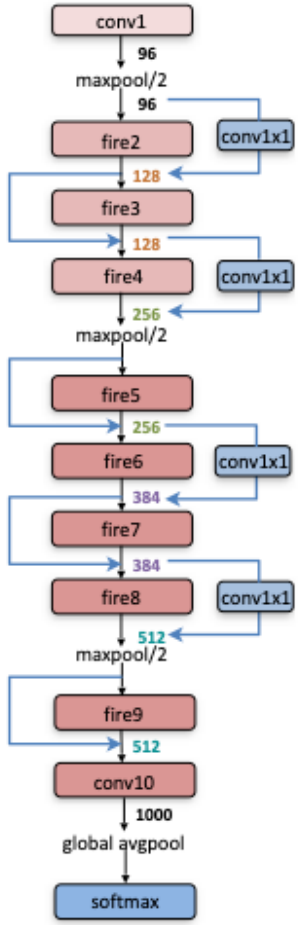


$$SR = \frac{|F_{s1x1}|}{|F_{e1x1}| + |F_{e3x3}|}$$

Controls how much to bottleneck

$$PCT_{3x3} = \frac{|F_{e3x3}|}{|F_{e1x1}| + |F_{e3x3}|}$$

Controls num filter params
(how many 1x1 versus 3x3)



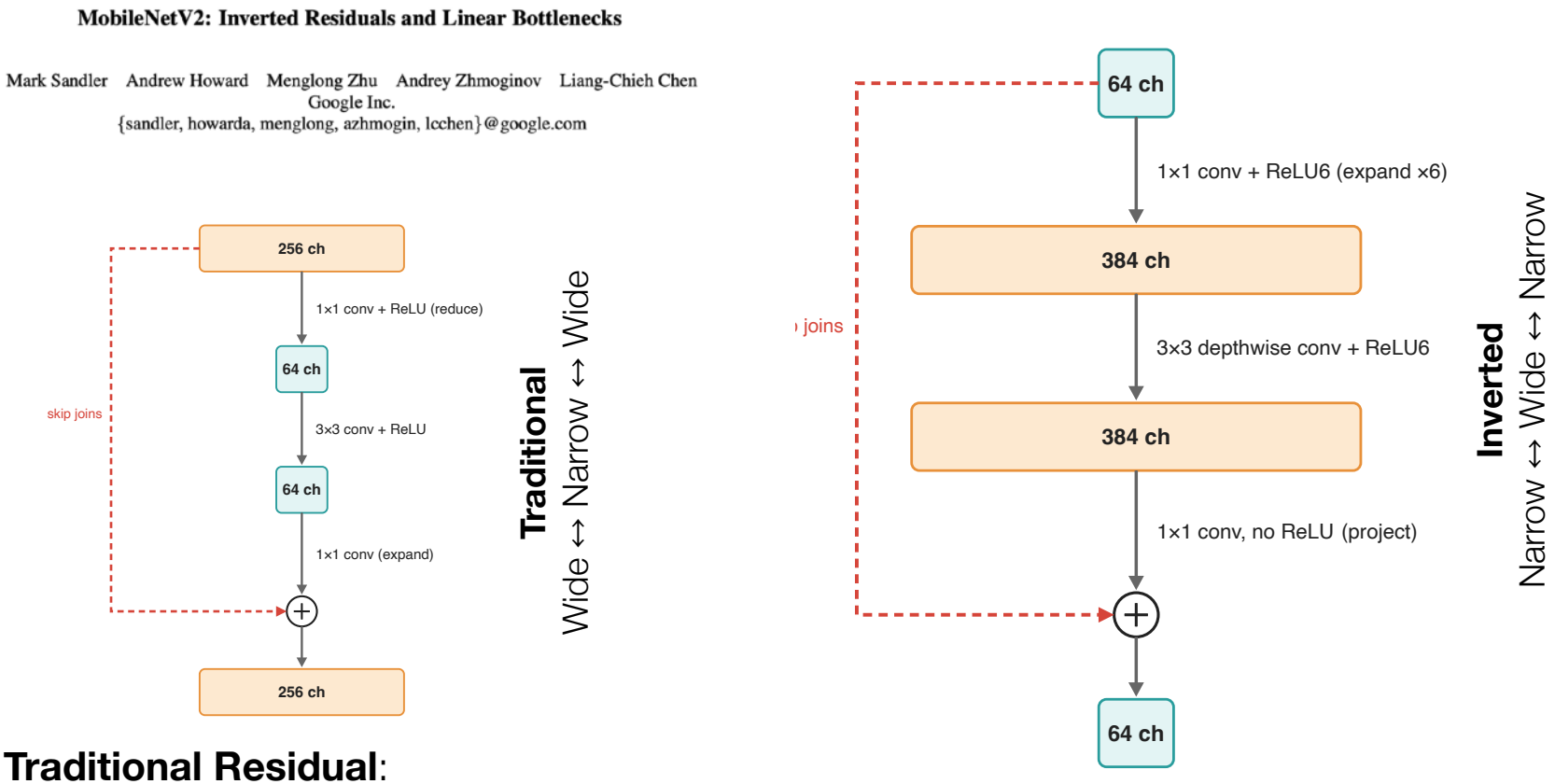
SQUEEZENET: ALEXNET-LEVEL ACCURACY WITH 50X FEWER PARAMETERS AND <0.5MB MODEL SIZE

Forrest N. Iandola[1], Song Han[2], Matthew W. Moskewicz[1], Khalid Ashraf[1], William J. Dally[2], Kurt Keutzer[1]
[1]DeepScale* & UC Berkeley [2]Stanford University
{forresti, moskewcz, kashraf, keutzer}@eecs.berkeley.edu
{songhan, dally}@stanford.edu

In paper, Good Performance when:

- SR is 12.5% up to 100%
- PCT_{3x3} is 25% up to 100%

MobileNet (2018), Wide/Narrow (num filters)



Traditional Residual:

- residual across wide tensors, expensive 3x3 conv runs on narrow reduction of channels

Inverted Residual:

- residual runs on narrow tensors, smaller stored activations and smaller sensitivity
- separable convolution is cheap, so spatial convolution is OK in high dimensional space
- because we reduce to narrower space, no need to have activation function (activation typically selects information, we just want the projection)

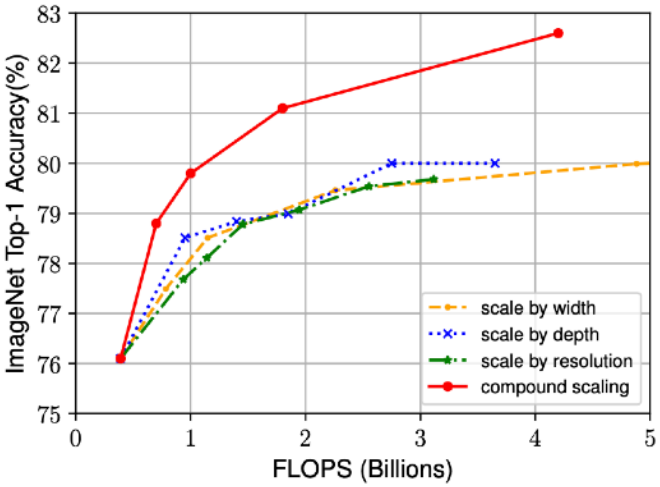
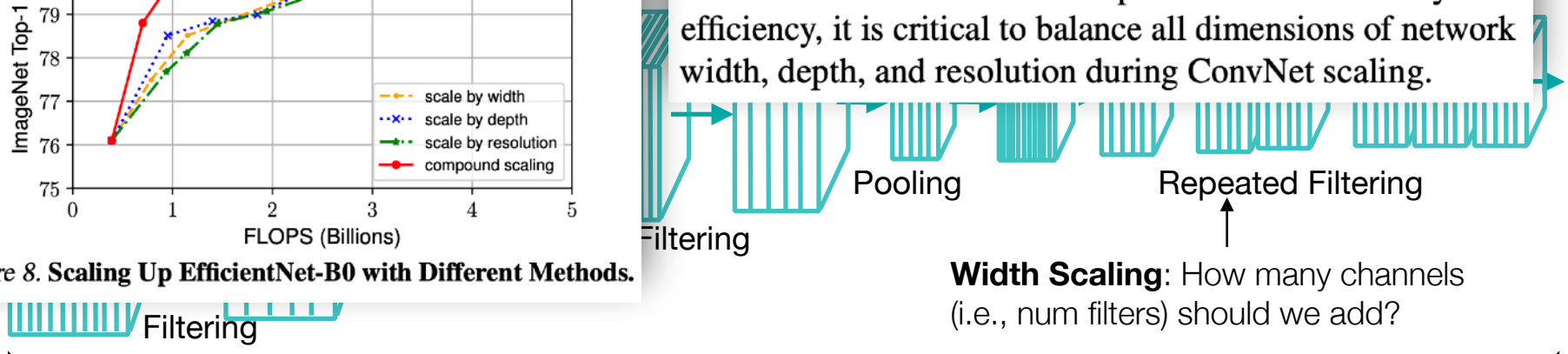


Figure 8. **Scaling Up EfficientNet-B0 with Different Methods.**

Observation 1 – Scaling up any dimension of network width, depth, or resolution improves accuracy, but the accuracy gain diminishes for bigger models.

Observation 2 – In order to pursue better accuracy and efficiency, it is critical to balance all dimensions of network width, depth, and resolution during ConvNet scaling.



Width Scaling: How many channels (i.e., num filters) should we add?

Depth Scaling: If we add layers, how should this scale with other parameters?

Resolution Scaling: For different resolutions, how should we scale the filters and layers?

depth: $d = \alpha^{\phi}$

width: $w = \beta^{\phi}$

res.: $r = \gamma^{\phi}$

s.t. $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$

$\alpha, \beta, \gamma \geq 1$

ϕ user specified scaling coefficient

$$\alpha = 1.2$$
$$\beta = 1.1$$
$$\gamma = 1.15$$

optimal values found in paper for $\phi = 1$ (via grid search, not theory)

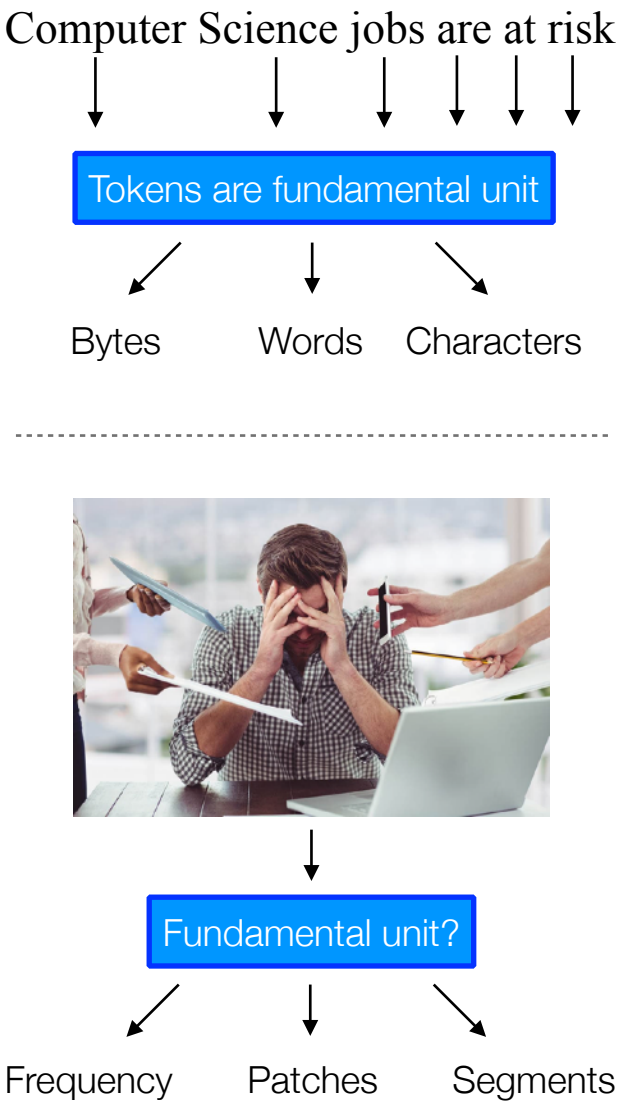
where α, β, γ are constants that can be determined by a small grid search. Intuitively, ϕ is a user-specified coefficient that controls how many more resources are available for model scaling, while α, β, γ specify how to assign these extra resources to network width, depth, and resolution re-

$\phi = 1$, keep FLOPS ~same :: $\phi = 2$, ~double the FLOPS from baseline

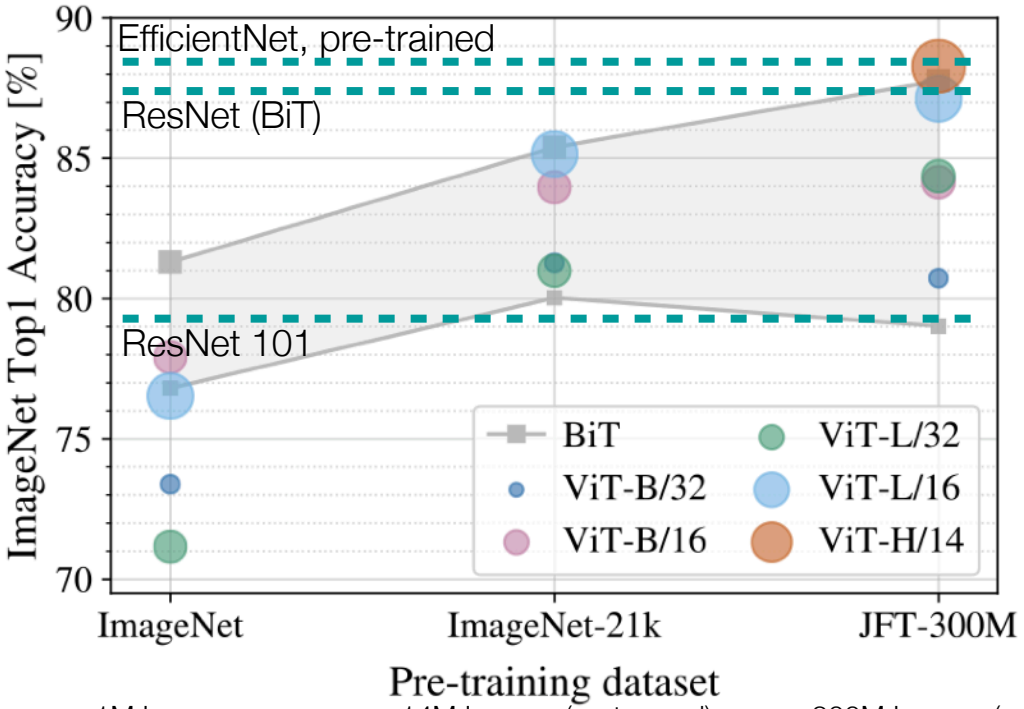
https://arxiv.org/pdf/1905.11946v5.pdf

Aside: Vision Transformers

- Transformers are great at attending to different tokens in language
- If we can define an image in terms of fundamental tokens and encode the image, we should be able to use attention
  - Fundamental token for an image?
  - What kind of pre-processing to capture token efficiently?
- Start simple, a token is a **patch** in an **image**
  - JPEG compression uses 8x8 tiles
  - Smaller patches would result in large sequence length, which is not good for a transformer
- 512x512 with 8x8 patch = 64x64 =4,096 sequence
- …but 16x16 would be 32x32 =1,024



Aside: The Vision Transformer



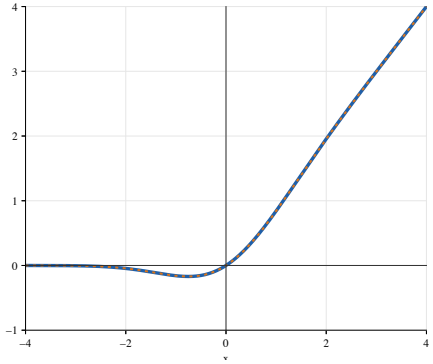
~1M Images
1k classes

~14M Images (part. crawl)
21k classes

~300M Images (crawled)
18k classes

CNNs that Compete with Vision Transformers

- **ConvNext** is an architecture that competes and many times beats out transformers, **but is just convolution** using some simple ideas make similar to x-former
  - (**new**) Patchify: make 4x4 conv with stride of 4 (patch)
  - Process these on four sets of resolutions (downsampling)
  - Inverted Residuals (1x1 to 4x4 then back 1x1)
  - (**new**) Layer normalization
  - (**new**) Gaussian Error Linear Unit (GELU) $x \cdot P(X \leq x) = x \cdot \Phi(x)$



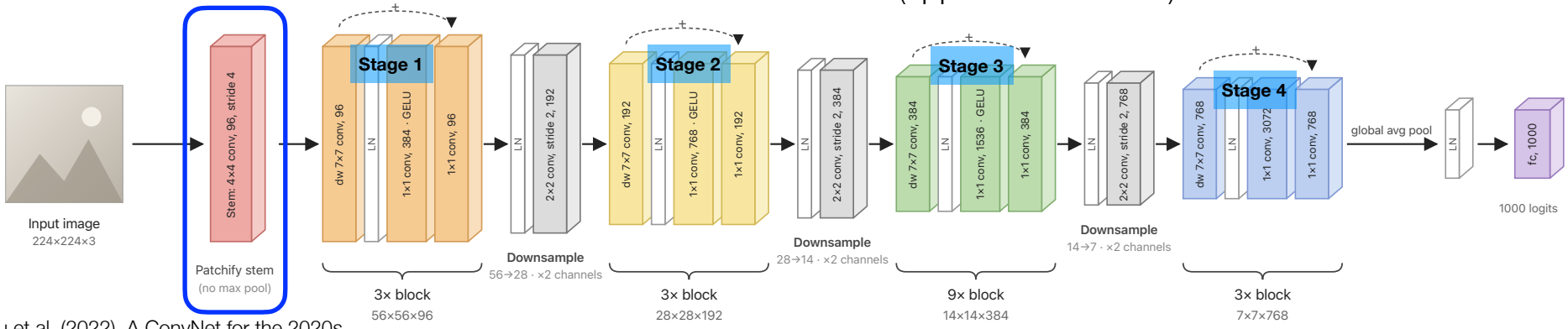
$$\mu = \frac{1}{C}\sum_{c=1}^{C} x_c \qquad \sigma^2 = \frac{1}{C}\sum_{c=1}^{C}(x_c - \mu)^2$$

$$\hat{x}_c = \frac{x_c - \mu}{\sqrt{\sigma^2 + \epsilon}} \qquad y_c = \gamma_c \hat{x}_c + \beta_c$$

$$\text{GELU}(x) = 0.5\,x\left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

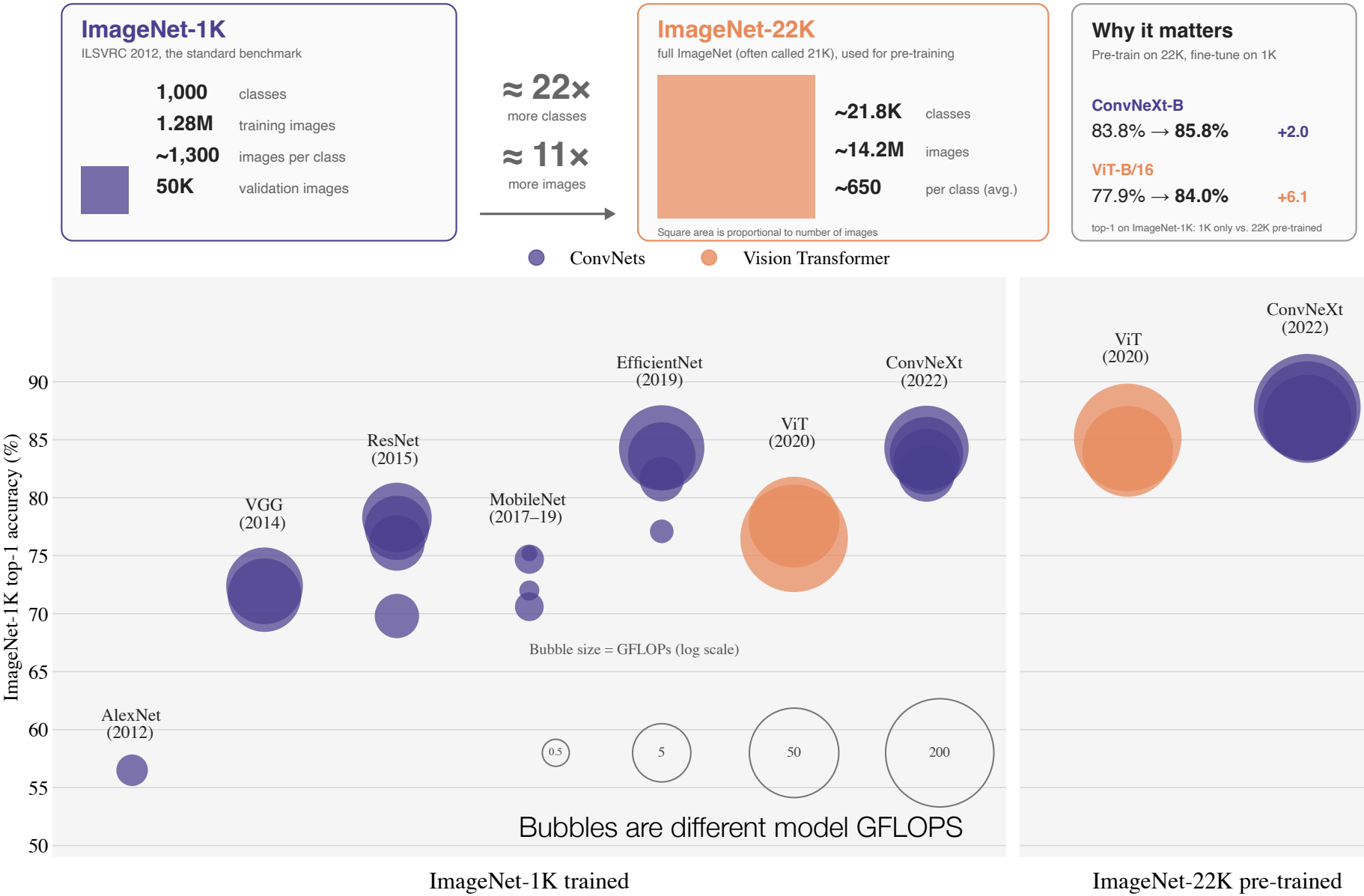
$$\text{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715\,x^3\right)\right]\right)$$

nn.GELU(approximate='tanh')



Liu et al. (2022), A ConvNet for the 2020s

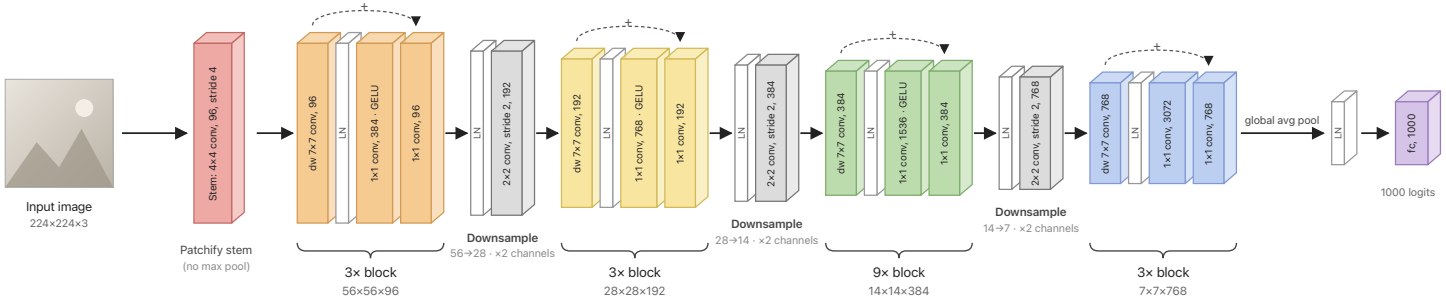
Performance Summary



Multiple bubbles per model = different model sizes. Top-1 from the original papers (MobileNet, EfficientNet, ViT, ConvNeXt) and torchvision (AlexNet, VGG, ResNet). ViT on 1K is trained from scratch.

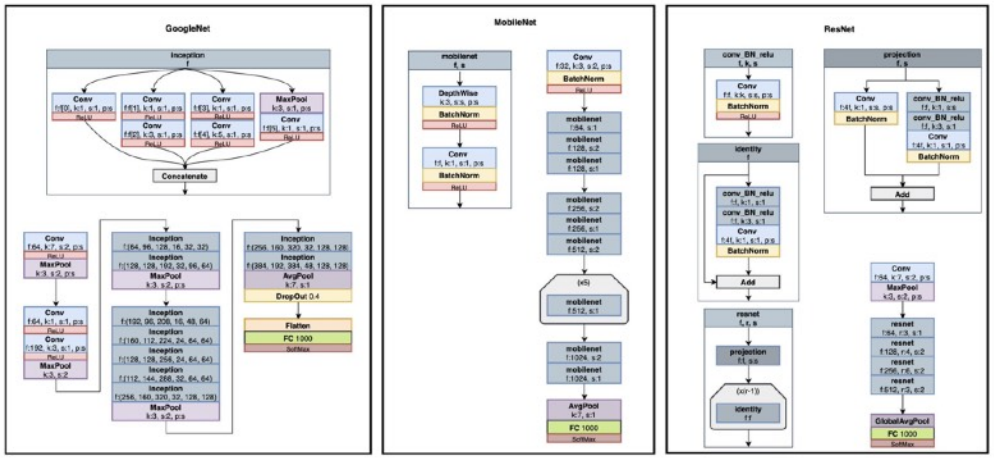
Self Test:

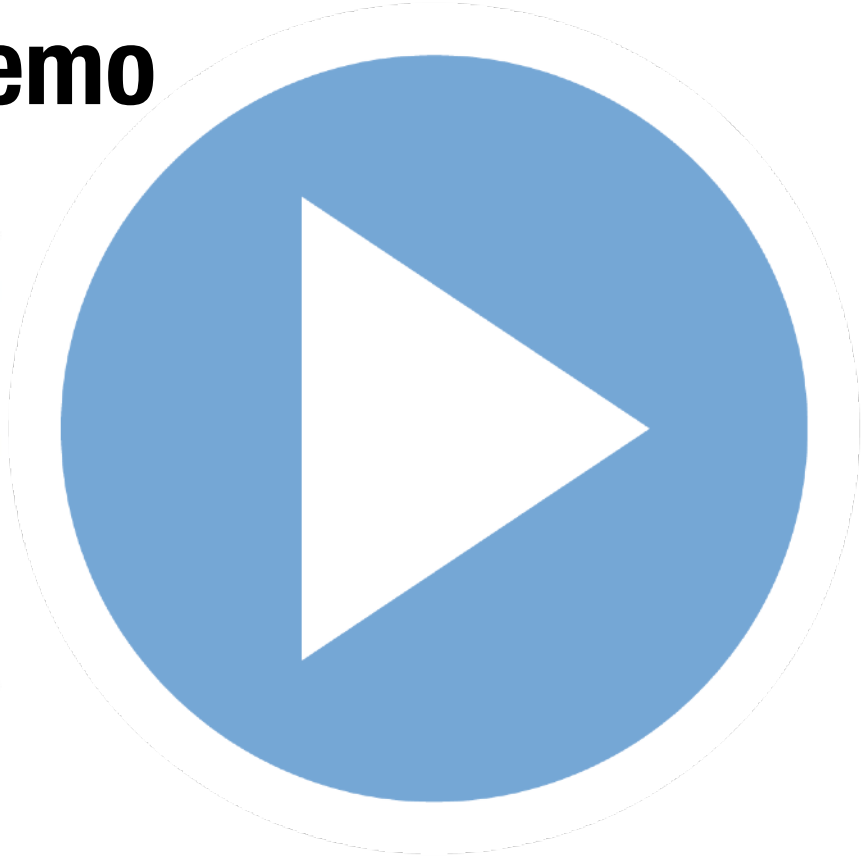
- **True/False**: ConvNext borrow ideas only from transformers and convolutional networks:
  - **A. True**. It uses an initial convolution that mirror patches from vision transformers
  - **B. True**. It borrows the idea of residual paths from CNNs (and transformers, which have not yet discussed)
  - **C. False**. It also borrows ideas from traditional neural networks like GeLU activations and layer normalization
  - **D. False**. It does not employ convolution.



Even more Convolutional Neural Networks

Mostly Self Guided Demo





Transfer Learning Also!

`10. PyTorch More Advanced CNN Techniques.ipynb`

Next Time:

- Intro to Sequential Neural Network Architectures
  - Word Embeddings, Transformers